

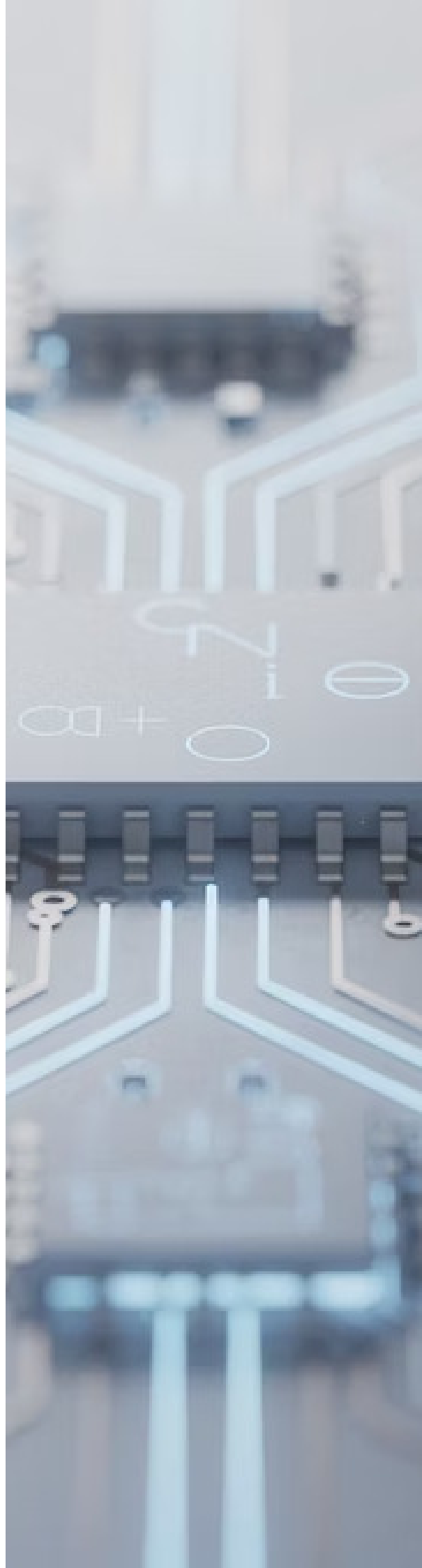
Función Booleana: Variables, Literales y Formas Normales

Una introducción a la representación de problemas lógicos mediante funciones booleanas, formas canónicas, compuertas lógicas y su implementación en hardware mediante circuitos digitales.

LÓGICA DIGITAL

ÁLGEBRA DE BOOLE

CIRCUITOS LÓGICOS



¿Qué es una Función Booleana?

Una **Función Booleana** es una expresión matemática que toma **n** variables de entrada (cada una valiendo 0 o 1) y produce una única salida (0 o 1).

Matemáticamente se escribe: $f : \{0, 1\}^n \rightarrow \{0, 1\}$

- Una función booleana de **n** variables tiene 2^n posibles combinaciones de entrada, y puede representar $2^{(2^n)}$ funciones distintas.

Variables y Literales

Variable

Entidad lógica que puede tomar únicamente los valores 0 o 1.

Literal afirmado

Representación de la variable tal cual (ej. A).

Literal negado

Representación de la variable invertida (ej. A').

Imagina una "caja negra" de decisiones.

Entradas del Sistema

- Sensor de puerta (A)
- Sensor de movimiento (B)
- Alarma activada (C)

Función F(A, B, C)

Determina si suena la sirena.

Salida

1 (Suena) o 0 (Silencio).

Tabla de Verdad: Sirena de Alarma

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Representación mediante Tablas de Verdad

Una función booleana de **n** variables puede representarse completamente mediante una tabla de verdad con 2^n **combinaciones** posibles de entradas. Cada fila corresponde a una combinación única de valores de las variables de entrada y muestra el valor de salida correspondiente.

1 variable
 $2^1 = 2$ filas

2 variables
 $2^2 = 4$ filas

3 variables
 $2^3 = 8$ filas

4 variables
 $2^4 = 16$ filas

Proceso de Construcción

Listar combinaciones
Escribir todas las combinaciones de entrada siguiendo el orden binario ascendente.

Evaluar la función
Calcular el valor de salida **f** para cada combinación de variables.

Identificar términos
Marcar los mintérminos (donde $f=1$) y los maxtérminos (donde $f=0$).

Ejemplo: $f(A,B,C) = AB + C'$

A	B	C	f
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Cómo leer la tabla

La tabla de verdad es la herramienta base para obtener las formas canónicas:

- Forma SOP (Sum of Products):** Se obtiene identificando las filas donde **f=1**; estas representan los *mintérminos* de la función.
- Forma POS (Product of Sums):** Se obtiene analizando las filas donde **f=0**; estas representan los *maxtérminos*.

Limitaciones del Álgebra de Boole

La herramienta fundamental para la simplificación de funciones booleanas es y seguirá siendo el álgebra de Boole; sin embargo, como ya lo ha mostrado la experiencia, el álgebra de Boole tiene las siguientes desventajas:

1. No es un método sistemático

No hay un algoritmo paso a paso a seguir.

2. Dificultad para verificar la reducción

No es fácil saber cuándo la expresión ya está lo más reducida posible.

3. Propenso a errores

Es fácil cometer errores y es difícil revisar el procedimiento.

Ejemplo de dificultad en simplificación

Consideremos la función $f = A'B'C + A'BC + AB'C + ABC$. A simple vista, no es obvio que esta expresión se reduce simplemente a $f = C$. El proceso algebraico es:

$$f = A'C(B' + B) + AC(B' + B) = A'C(1) + AC(1) = C(A' + A) = C(1) = C$$

Este ejemplo demuestra cómo la manipulación algebraica requiere identificar factores comunes y aplicar identidades booleanas de forma inteligente, lo cual no siempre es intuitivo.

Métodos alternativos

1 Mapa de Karnaugh (K-map)

Método gráfico sistemático muy intuitivo para visualización.

2 Método de Quine-McCluskey

Algoritmo tabular ideal para funciones con un gran número de variables.

3 Herramientas CAD

Síntesis lógica automatizada utilizada en el diseño de circuitos modernos.

Por ello, es importante contar con un método sistemático y gráfico, que sea más sencillo y poderoso para la simplificación de funciones.

📌 **Nota:** El Mapa de Karnaugh es el método gráfico más utilizado en la práctica para funciones de hasta 6 variables.

📌 Sin embargo, previamente a la presentación del método se requieren algunas definiciones que se usarán de aquí en adelante.

Formas Canónicas: Término Producto y Forma SOP

Término Producto. Se llama término producto una expresión booleana que solamente incluye operaciones AND entre sus variables (afirmadas o negadas).

Ejemplos de términos producto:

$AB', ABC', BC, A'BCD', B$, etc.

Forma SOP. Una función booleana se dice que está en la forma de suma de productos (SOP) si está formada exclusivamente por la suma (OR) de términos producto.

Ejemplos de funciones en forma SOP son:

$$f_1(A, B, C, D) = AB'C + B'D' + A'CD'$$

$$f_2(X, Y, Z) = XZ' + YZ' + XYZ$$

Operación base

AND entre variables

Combinación

OR entre términos producto

Variables

Afirmadas o negadas

Comparación de Formas

Característica	Término Producto	Forma SOP
Operación base	AND	OR de términos AND
Número de variables	Parcial o completa	Parcial o completa
Ejemplo	ABC'	$AB + C$

Verificación de forma SOP

Dada la función $f = AB'C + B'D' + A'CD'$:

- Verificación de términos:** Cada término ($AB'C$, $B'D'$, $A'CD'$) consiste únicamente en variables conectadas por AND.
- Verificación de la función:** Los términos están combinados mediante OR.
- Nota sobre canonicidad:** Esta forma **NO es canónica**, ya que los términos no contienen todas las variables del sistema (por ejemplo, el primer término no incluye la variable D).

Propiedades clave de la forma SOP

→ Implementación Fácil de implementar con compuertas AND-OR de dos niveles.	→ Relación con minterminos Directamente relacionada con los minterminos de la función.	→ Simplificación Permite simplificación mediante álgebra de Boole o Mapas de Karnaugh.
---	--	--

Mintérminos y Forma Canónica SOP

Mintérmino. Es un término producto que contiene **todas las variables** de la función.

Ejemplos de mintérminos para una función de 4 variables A, B, C, D:

$ABCD, AB'CD', A'BCD, \text{etc.}$

Forma Canónica SOP. Si los términos producto de una función booleana en la forma SOP son todos mintérminos, se dice que está en la **forma canónica SOP**.

Ejemplos de funciones en forma canónica SOP son:

$$f_3(A, B, C) = \bar{A}B\bar{C} + AB\bar{C} + \bar{A}BC + ABC$$
$$f_4(A, B, C, D, E) = \bar{A}BC\bar{D}E + ABC\bar{D}E + \bar{A}BCDE$$

☐ La forma canónica SOP se basa en los **mintérminos**, es decir, las combinaciones de entrada donde la función vale 1.

Tabla de Mintérminos (3 variables: A, B, C)

Índice	A	B	C	Mintérmino	Notación
0	0	0	0	A'B'C'	m0
1	0	0	1	A'B'C	m1
2	0	1	0	A'BC'	m2
3	0	1	1	A'BC	m3
4	1	0	0	AB'C'	m4
5	1	0	1	AB'C	m5
6	1	1	0	ABC'	m6
7	1	1	1	ABC	m7

Cómo obtener la Forma Canónica SOP desde la Tabla de Verdad

- Identificar todas las filas de la tabla de verdad donde la función $f = 1$.
- Escribir el mintérmino correspondiente a cada una de esas filas.
- Unir todos los mintérminos obtenidos mediante la operación OR (suma).

Ejemplo Concreto

Dada una función $f(A, B, C)$ que es igual a 1 en las filas 1, 3, 5 y 7:

$$f = m_1 + m_3 + m_5 + m_7$$
$$f = A'B'C + A'BC + AB'C + ABC$$

Al simplificar esta expresión mediante álgebra booleana:

$$f = A'C(B' + B) + AC(B' + B) = A'C(1) + AC(1) = C(A' + A) = C(1) = C$$

Término Suma, Forma POS, Maxtérminos y Forma Canónica POS

Término Suma. Se llama término suma a una expresión booleana que solamente incluye operaciones OR entre sus variables (afirmadas o negadas).

Ejemplos de términos suma:

$A + B', A + B + C', B + C, A' + B + C + D'$

Forma POS. Una función booleana se dice que está en la forma de producto de sumas (POS) si está formada exclusivamente por el producto (AND) de términos suma.

Ejemplos de funciones en forma POS son:

$$f_5(A, B, C, D) = (A + B' + C)(B' + D')(A' + C + D')$$
$$f_6(X, Y, Z) = (X + \bar{Z})(Y + \bar{Z})(X + Y + Z)$$

Maxtérminos. Son términos suma que contienen **todas las variables** de la función.

Ejemplos de maxtérminos para una función de 4 variables A, B, C, D son:

$A + B + C + D, A + B + C' + D', A' + B + C + D$

Forma Canónica POS. Si los términos suma de una función booleana en la forma POS son todos maxtérminos, se dice que está en la **forma canónica POS**.

Ejemplos de funciones en forma canónica POS son:

$$f_7(A, B, C) = (\bar{A} + B + \bar{C})(A + B + \bar{C})(A + B + C)$$
$$f_8(A, B, C, D, E) = (A + B' + C + D')(A + B + C' + D' + E)(A' + B + C + D + E)$$

☐ La forma canónica POS se basa en los **maxtérminos**, es decir, las combinaciones de entrada donde la función vale **0**.

Tabla de Maxtérminos (3 variables: A, B, C)

Índice	A	B	C	Maxtérmino	Notación
0	0	0	0	A+B+C	M0
1	0	0	1	A+B+C'	M1
2	0	1	0	A+B'+C	M2
3	0	1	1	A+B'+C'	M3
4	1	0	0	A'+B+C	M4
5	1	0	1	A'+B+C'	M5
6	1	1	0	A'+B'+C	M6
7	1	1	1	A'+B'+C'	M7

Relación entre mintérminos y maxtérminos

Los mintérminos (m_i) y los maxtérminos (M_i) son complementos entre sí, lo cual se expresa mediante la relación:

$m_i = \bar{M}_i$

Esto significa que una función puede representarse mediante la suma de los mintérminos donde la función es 1, o mediante el producto de los maxtérminos donde la función es 0:

$$f = \sum m(1, 3, 5, 7) = \prod M(0, 2, 4, 6)$$

☐ Regla práctica: los índices de los mintérminos y los maxtérminos de una misma función son complementarios (suman el conjunto completo de 2^n índices).

Notación: Lista de Mintérminos y Maxtérminos

Una manera de simplificar la escritura de las funciones en forma canónica consiste en representar sus términos por números binarios, en base a la siguiente convención:

	Mintérmino	Maxtérmino
Variable afirmada	1	0
Variable negada	0	1

Notación Σm (lista)
Suma de mintérminos: se listan los índices donde f=1.

Notación ΠM (lista)
Producto de maxtérminos: se listan los índices donde f=0.

Por ejemplo, usando esta notación la función $f_3(A, B, C)$ puede escribirse como:

$$f_3(A, B, C) = A'BC' + ABC' + A'BC + ABC = 010 + 110 + 011 + 111 = m_2 + m_6 + m_3 + m_7$$

O en forma más compacta: $f_3(A, B, C) = \sum m(2, 3, 6, 7)$

En forma similar, la función $f_7(A, B, C)$ puede escribirse como:

$$f_7(A, B, C) = (A' + B + C')(A + B + C')(A' + B + C)(A + B + C) = (101)(001)(100)(000) = M_5 \ M_1 \ M_4 \ M_0$$

O en forma más compacta: $f_7(A, B, C) = \prod M(0, 1, 4, 5)$

☐ **Observación:** El orden de las variables en la notación anterior debe coincidir con el orden en que están especificadas, es decir, $f(A, B, C) \neq f(B, C, A) \neq f(A, C, B)$, etc.

Ejercicio resuelto completo

Dada la función $f(A, B, C)$ tal que $f = 1$ cuando $C = 0$ (filas 0, 2, 4, 6), obtenemos:

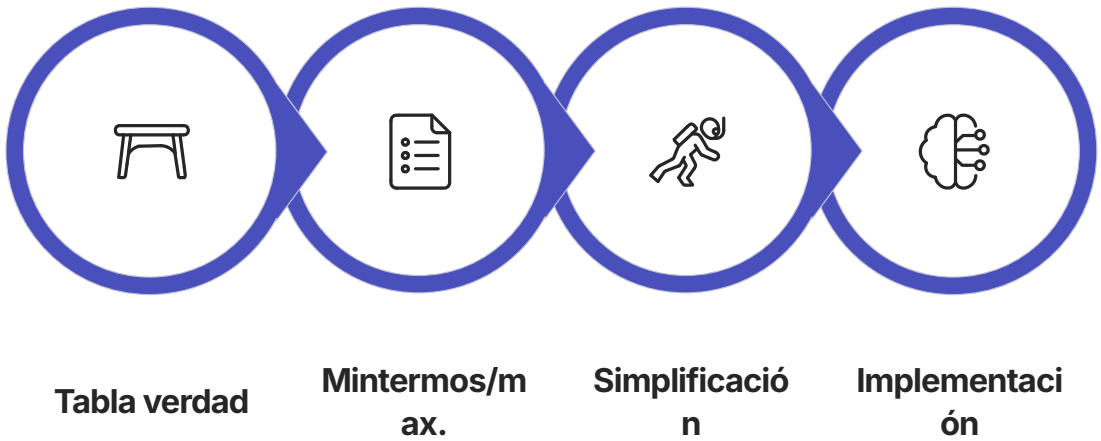
$$f = \sum m(0, 2, 4, 6) = \prod M(1, 3, 5, 7)$$

A	B	C	f	Índice	Tipo
0	0	0	1	0	m0
0	0	1	0	1	M1
0	1	0	1	2	m2
0	1	1	0	3	M3
1	0	0	1	4	m4
1	0	1	0	5	M5
1	1	0	1	6	m6
1	1	1	0	7	M7

☐ Verificación: la unión de los índices de mintérminos y maxtérminos siempre debe dar el conjunto completo $\{0, 1, ..., 2^n - 1\}$.

Procedimiento para Obtener Expresiones desde la Tabla de Verdad

A partir de los conceptos y definiciones anteriores ya estamos en condiciones de plantear algunos diseños sencillos de circuitos lógicos. Podemos ordenar el procedimiento para esto de acuerdo a los siguientes pasos:



Este procedimiento sistemático permite pasar de la descripción verbal de un problema lógico hasta su implementación física en hardware mediante compuertas lógicas.

01	02
Planteamiento en tabla de verdad	Obtención de la forma canónica
Definir la función que debe hacer el circuito.	Lista de minterminos o de maxtérminos.
03	04
Simplificación de la función lógica	Implementación con compuertas
Reducir la expresión al mínimo posible.	Realizar el circuito con compuertas lógicas.

Ejemplo aplicado al procedimiento

Diseñar un circuito que detecte si un número binario de 2 bits (A, B) es mayor o igual a 2 (es decir, el valor decimal es 2 o 3).

Paso 1: Tabla de verdad

A	B	f
0	0	0
0	1	0
1	0	1
1	1	1

Paso 2: Forma canónica

Identificamos los minterminos donde f=1 (índices 2 y 3):

$$f = \sum m(2,3) = AB' + AB$$

Paso 3: Simplificación

Utilizando álgebra de Boole:

$$f = AB' + AB = A(B' + B) = A(1) = A$$

Paso 4: Implementación

Como el resultado es $f = A$, el circuito no requiere compuertas lógicas; basta con conectar la entrada A directamente a la salida.

Observación: Este ejemplo muestra que la simplificación puede reducir drásticamente el hardware necesario: de 2 minterminos a una sola variable.

Compuertas Lógicas Básicas: AND, OR, NOT

Las compuertas lógicas son los bloques fundamentales de construcción de cualquier circuito digital. Las compuertas básicas son:



AND

Salida = 1 solo si **todas** las entradas son 1. Implementa el producto lógico ($\cdot$).



OR

Salida = 1 si **al menos una** entrada es 1. Implementa la suma lógica ($+$).



NOT

Salida = complemento de la entrada. Implementa la negación ($'$).

AND

A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

OR

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT

A	A'
0	1
1	0

Con estas tres compuertas es posible implementar **cualquier función booleana**. La forma SOP utiliza compuertas AND para los términos producto y una compuerta OR para sumarlos; la forma POS utiliza compuertas OR para los términos suma y una compuerta AND para multiplicarlos.

Implementación de SOP con AND-OR

Una función en forma SOP se implementa directamente con un nivel de compuertas AND (una por cada mintermino) seguido de una compuerta OR final. Por ejemplo, la función $f = AB + A'C$ requiere:

- 2 compuertas AND de 2 entradas (para los términos AB y A'C).
- 1 compuerta OR de 2 entradas (para sumar los resultados).

 **Observación:** Toda función booleana puede implementarse en exactamente 2 niveles de lógica usando la forma SOP (AND-OR) o la forma POS (OR-AND).

Compuertas Universales: NAND y NOR

Las compuertas **NAND** y **NOR** se denominan **compuertas universales** porque con cualquiera de ellas sola es posible implementar cualquier función booleana, sin necesidad de las otras compuertas básicas.

Compuerta NAND

NOT-AND: Su salida es el complemento de la AND. Salida = 0 solo si **todas** las entradas son 1.

$$\overline{A \cdot B}$$

Compuerta NOR

NOT-OR: Su salida es el complemento de la OR. Salida = 1 solo si **todas** las entradas son 0.

$$\overline{A + B}$$

Tabla de verdad NAND

A	B	(A·B)'
0	0	1
0	1	1
1	0	1
1	1	0

Tabla de verdad NOR

A	B	(A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

Implementación de NOT, AND y OR con solo NAND

- NOT A: $A' = (A \cdot A)'$
- AND: $A \cdot B = ((A \cdot B)')'$
- OR: $A + B = (A' \cdot B')' = ((A \cdot A)' \cdot (B \cdot B)')'$

Implementación de NOT, AND y OR con solo NOR

- NOT A: $A' = (A + A)'$
- OR: $A + B = ((A + B)')'$
- AND: $A \cdot B = (A' + B')' = ((A + A)' + (B + B)')'$



Observación: En la industria, las compuertas NAND son preferidas porque son más fáciles y económicas de fabricar en tecnología CMOS.

Compuertas Aritméticas: XOR y XNOR

Además de las compuertas básicas y universales, existen compuertas aritméticas de gran utilidad en el diseño de circuitos digitales:

1

XOR (OR Exclusivo)
Salida = 1 si las entradas son diferentes. Salida = 0 si las entradas son iguales. Expresión: $A \oplus B = A'B + AB'$. Fundamental en sumadores, comparadores y circuitos de paridad.

2

XNOR (NOR Exclusivo)
Salida = 1 si las entradas son iguales. Salida = 0 si las entradas son diferentes. Expresión: $(A \oplus B)' = AB + A'B'$. Utilizada en comparadores de igualdad y detectores de coincidencia.

Observación: XOR y XNOR son el complemento una de la otra: $XNOR = XOR'$.

Tablas de Verdad

Tabla XOR

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Tabla XNOR

A	B	$(A \oplus B)'$
0	0	1
0	1	0
1	0	0
1	1	1

Aplicaciones prácticas

Sumador de 1 bit: XOR genera el bit de suma, AND genera el acarreo

Detector de paridad: XOR de n bits detecta si hay número impar de 1s

Comparador de igualdad: XNOR compara bit a bit

Generador de complemento a 2: XOR con 1 invierte bits selectivamente

Semisumador (Half Adder)

Suma = $A \oplus B$, Acarreo = $A \cdot B$.

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Observación: El semisumador es el bloque básico de todos los circuitos aritméticos digitales.

Ejemplo: Circuito del Jurado

Un jurado está formado por tres jueces A, B, C, cada juez emite su voto a favor oprimiendo un botón enfrente de él. Se desea construir un circuito que encienda una luz que indique si la mayor parte del jurado votó a favor y no la encienda en cualquier otro caso.

Tabla de Verdad

A	B	C	L (Luz)
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Función Booleana

La función que realiza lo deseado es:

$$L(A, B, C) = \sum m(3, 5, 6, 7)$$

Interpretación

La luz enciende cuando al menos **dos de los tres jueces** votan a favor (mayoría simple).

Obtención de la forma canónica SOP

Identificamos los minitérminos donde la salida es 1:

- m3: A=0, B=1, C=1 → A'BC
- m5: A=1, B=0, C=1 → AB'C
- m6: A=1, B=1, C=0 → ABC'
- m7: A=1, B=1, C=1 → ABC

$$L = A'BC + AB'C + ABC' + ABC$$

Simplificación algebraica paso a paso

Para simplificar, utilizamos la propiedad idempotente $X + X = X$ para duplicar ABC:

$$L = (A'BC + ABC) + (AB'C + ABC) + (ABC' + ABC)$$

Agrupamos términos para eliminar variables:

$$L = BC(A' + A) + AC(B' + B) + AB(C' + C)$$

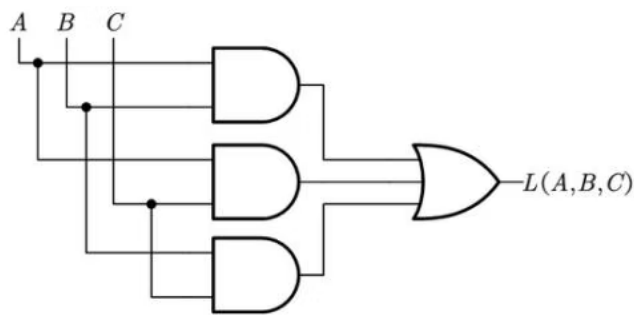
Como $A' + A = 1$, $B' + B = 1$ y $C' + C = 1$, obtenemos el resultado final:

$$L = BC + AC + AB$$

 **Observación:** La función $L = AB + AC + BC$ es la función mayoría de 3 variables: es 1 cuando al menos 2 de las 3 variables son 1.

Síntesis del Circuito: De la Expresión a las Compuertas

A partir de la función simplificada $L(A,B,C) = AB + AC + BC$, se implementa el circuito lógico correspondiente. La síntesis consiste en asegurar la **correspondencia lógica entre la función y el hardware**.



El circuito consiste en **tres compuertas AND** de 2 entradas y **una compuerta OR** de 3 entradas:

1

AND 1
Entradas: A, B → Producto AB

2

AND 2
Entradas: A, C → Producto AC

3

AND 3
Entradas: B, C → Producto BC

4

OR Final
Suma $AB + AC + BC$ → Salida L

Conteo de compuertas y niveles lógicos

Análisis de complejidad estructural del circuito:

3

Compuertas AND

1

Compuerta OR

2

Niveles Lógicos

6

Entradas Totales

Implementación alternativa con NAND

Utilizando el teorema de De Morgan, podemos transformar la función SOP a una forma universal basada únicamente en compuertas NAND:

$$L = AB + AC + BC = ((AB)' \cdot (AC)' \cdot (BC)')'$$

Esta implementación permite utilizar la tecnología de fabricación estándar (CMOS), donde las compuertas NAND son más eficientes.

- ☐

Observación: La implementación con NAND requiere el mismo número de compuertas (4 NAND) pero es preferida en la práctica por la uniformidad tecnológica.
- ☐

Como habrá podido advertirse, en el procedimiento anterior, los únicos pasos que no han sido completamente sistematizados son el 1 y el 3 (la simplificación de la función).

Resumen: Ejes Temáticos del Bloque

Este bloque aborda la representación de problemas lógicos mediante funciones booleanas y su implementación en hardware mediante compuertas lógicas.

1

Funciones Booleanas

Definición de $f : B^n \rightarrow B$. Representación mediante tablas de verdad de n variables (2^n combinaciones).

2

Formas Canónicas

SOP: Basada en mintérminos (donde la función es 1). **POS:** Basada en maxtérminos (donde la función es 0). Procedimiento para obtener expresiones a partir de la tabla de verdad.

3

Compuertas Lógicas

Análisis de compuertas básicas (AND, OR, NOT) y compuertas universales (NAND, NOR). Inclusión de compuertas aritméticas como XOR y XNOR.

4

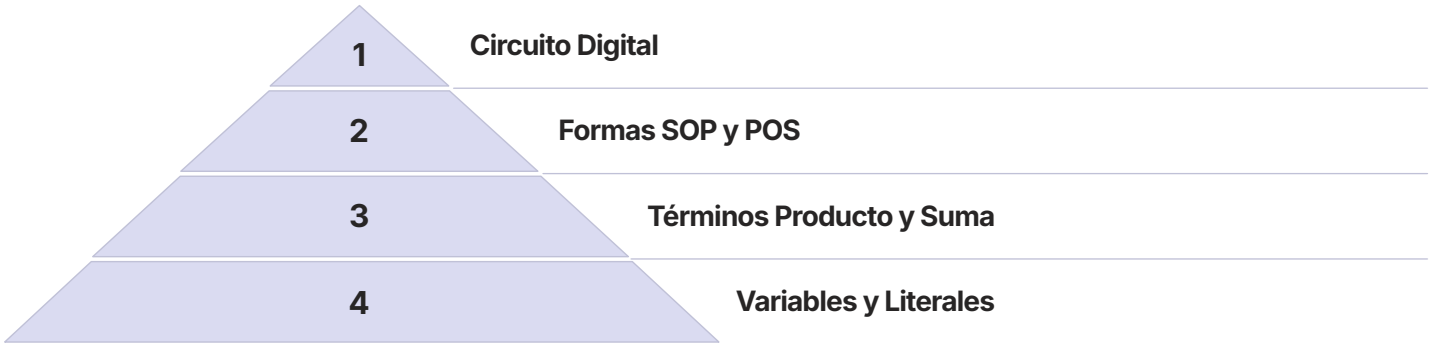
Síntesis de Circuitos

De la expresión booleana al diagrama de compuertas, asegurando la correspondencia lógica entre la función y el hardware.

Conceptos clave y sus definiciones

Concepto	Definición	Notación
Variable booleana	Toma valores 0 o 1	A, B, C...
Literal	Variable afirmada o negada	A, A'
Mintérmino	Término producto con todas las variables	m_i
Maxtérmino	Término suma con todas las variables	M_i
Forma SOP	Suma de productos	$\Sigma m(\dots)$
Forma POS	Producto de sumas	$\Pi M(\dots)$

Jerarquía de conceptos



El dominio de estos conceptos es la base para el diseño sistemático de cualquier circuito digital combinacional.